

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import zipfile
import os

# Define the path to your zip file and the extraction directory
# Replace 'path/to/your/fyp_Ai.zip' with the actual path to your zip file
# Replace 'path/to/extract/to' with the directory where you want to extract the contents
zip_file_path = '/content/drive/MyDrive/FYP_AI-DRONES.1.zip' # Example for Google Drive
extraction_dir = 'fyp_Ai_extracted'

# Create the extraction directory if it doesn't exist
os.makedirs(extraction_dir, exist_ok=True)

# Check if the zip file exists
if os.path.exists(zip_file_path):
    try:
        with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
            zip_ref.extractall(extraction_dir)
        print(f"Successfully unzipped '{zip_file_path}' to '{extraction_dir}'")
    except zipfile.BadZipFile:
        print(f"Error: The file '{zip_file_path}' is not a valid zip file.")
    except Exception as e:
        print(f"An error occurred during unzipping: {e}")
else:
    print(f"Error: Zip file not found at '{zip_file_path}'")
```

Successfully unzipped '/content/drive/MyDrive/FYP_AI-DRONES.1.zip' to 'fyp_Ai_extracted'

```
!pip install ultralytics
```

```
Downloading ultralytics-8.4.17-py3-none-any.whl.metadata (39 kB)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.13.0)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.10.0+cu128)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (0.25.0+cu128)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.31.0)
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
  Downloading ultralytics_thop-2.0.18-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.22.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.4.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (24.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2025.1.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.14.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (4.11.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (75.1.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.4.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (2025.1.1)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (1.13.1.3)
Requirement already satisfied: cuda-nathfinder==1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->ultralytics) (1.1)
```

```
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.13.3->torch)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->ultralytics)
Downloading ultralytics-8.4.17-py3-none-any.whl (1.2 MB)
1.2/1.2 MB 31.9 MB/s eta 0:00:00
Downloading ultralytics_thop-2.0.18-py3-none-any.whl (28 kB)
Installing collected packages: ultralytics-thop, ultralytics
Successfully installed ultralytics-8.4.17 ultralytics-thop-2.0.18
```

Task

To resolve the "invalid class indices" error, I'll start by inspecting the label files to identify all unique class IDs. This will help confirm the existence of class IDs beyond the expected range of 0-4, as indicated by the error message. I'll read all the `.txt` label files in the training and validation directories: `content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/labels/train` and `content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/labels/val`. I will store the identified class IDs in a set to ensure uniqueness.

Identify All Class IDs

Subtask:

Scan all label (`.txt`) files in the training and validation directories to identify every unique class ID present. This will confirm the presence of class IDs beyond 0-4.

Reasoning: To identify all unique class IDs, I will define the paths to the training and validation label directories, create a helper function to read each label file, extract the class ID (the first integer on each line), and add it to a set. Finally, I will print the set of unique class IDs.

```
import os

# 1. Define the paths to the training and validation label directories
train_labels_dir = '/content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/labels/train'
val_labels_dir = '/content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/labels/val'

# 2. Initialize an empty set to store all unique class IDs found
unique_class_ids = set()

# 3. Create a helper function to process label files
def process_label_files(directory_path, unique_class_ids_set):
    if not os.path.exists(directory_path):
        print(f"Warning: Directory not found at {directory_path}")
        return

    for filename in os.listdir(directory_path):
        if filename.endswith('.txt'):
            file_path = os.path.join(directory_path, filename)
            try:
                with open(file_path, 'r') as f:
                    for line in f:
                        parts = line.strip().split()
                        if parts:
                            # 4b. For each line, extract the first integer (class ID)
                            class_id = int(parts[0])
                            # 4c. Add this class ID to the unique_class_ids_set
                            unique_class_ids_set.add(class_id)
            except Exception as e:
                print(f"Error processing file {file_path}: {e}")

# 5. Call the helper function for both training and validation label directories
print(f"Processing training labels in: {train_labels_dir}")
process_label_files(train_labels_dir, unique_class_ids)

print(f"Processing validation labels in: {val_labels_dir}")
process_label_files(val_labels_dir, unique_class_ids)

# 6. Print the unique_class_ids set
print("\nAll unique class IDs found:")
print(sorted(list(unique_class_ids)))
```

```
Processing training labels in: /content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/labels/train
Processing validation labels in: /content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/labels/val
```

```
All unique class IDs found:
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Update `visdrone_reduced.yaml` with all 10 class names

Now that we've identified all unique class IDs (0-9) present in the dataset, we need to update the `visdrone_reduced.yaml` file to reflect these 10 classes. The current `names` section only lists 5 classes, which is causing the 'invalid class indices' error during training.

We will update the `names` section to include the correct names for all 10 classes as typically defined in the VisDrone dataset:

- 0: pedestrian
- 1: person
- 2: bicycle
- 3: car
- 4: van
- 5: truck
- 6: tricycle
- 7: awning-tricycle
- 8: bus
- 9: motor

Reasoning: To resolve the 'invalid class indices' error, I will update the `visdrone_reduced.yaml` file by replacing the existing 'names' section with the complete list of 10 class names (0-9) identified in the dataset.

Reasoning: Now that the `visdrone_reduced.yaml` file has been updated with the correct 10 class names, the previous `KeyError` should be resolved. I will re-attempt the training using the same parameters.

Save and Download Project

Subtask:

Create a zip archive of the `fyp_Ai_extracted` directory, which contains all your processed data and training results, so you can easily download your entire project.

Reasoning: To enable easy download of the entire project, I will use the `zipfile` module to create a compressed archive of the `fyp_Ai_extracted` directory. This will include all modifications made to the dataset and any generated training artifacts.

```
import shutil
import os

# Define the directory to zip and the output zip file name
directory_to_zip = 'fyp_Ai_extracted'
output_zip_name = 'fyp_Ai_project'
output_zip_path = f'/content/{output_zip_name}'

print(f"Creating zip archive of '{directory_to_zip}'...")

try:
    # Create a zip archive
    shutil.make_archive(output_zip_path, 'zip', directory_to_zip)
    print(f"Successfully created zip file: {output_zip_path}.zip")
    print("You can now download this file from the Colab file browser (left sidebar -> 'Files' icon) or using the following command: cp {output_zip_path}.zip /content/drive/MyDrive/")
    print("After running the above command, the zip file will be copied to your Google Drive.")
except Exception as e:
    print(f"An error occurred while creating the zip file: {e}")
```

```
Creating zip archive of 'fyp_Ai_extracted'...
Successfully created zip file: /content/fyp_Ai_project.zip
You can now download this file from the Colab file browser (left sidebar -> 'Files' icon) or using the following command:
!cp /content/fyp_Ai_project.zip /content/drive/MyDrive/
After running the above command, the zip file will be copied to your Google Drive.
```

```
import os

yaml_file_path = '/content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/VisDrone.yaml'

# Desired content for the YAML file with all 10 classes and correct absolute paths
updated_yaml_content = """
train: /content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/images/train
```

```
val: /content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/images/val
```

```
names:
```

```
0: pedestrian
1: person
2: bicycle
3: car
4: van
5: truck
6: tricycle
7: awning-tricycle
8: bus
9: motor
```

```
"""
```

```
try:
```

```
# Write the updated content back to the YAML file
with open(yaml_file_path, 'w') as f:
    f.write(updated_yaml_content)
    print(f"Successfully updated '{yaml_file_path}' with the correct absolute paths and all 10 class names.")
    print("New YAML content:\n" + updated_yaml_content)
```

```
except Exception as e:
```

```
    print(f"An error occurred while writing to the YAML file: {e}")
```

Successfully updated '/content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/VisDrone.yaml' with the correct absolute paths and all 10 class names.
New YAML content:

```
train: /content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/images/train
```

```
val: /content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/images/val
```

```
names:
```

```
0: pedestrian
1: person
2: bicycle
3: car
4: van
5: truck
6: tricycle
7: awning-tricycle
8: bus
9: motor
```

```
import shutil
```

```
import os
```

```
# Define the source directory in Google Drive
```

```
source_dir_to_zip = '/content/drive/MyDrive/yolo_runs/visdrone_exp114'
```

```
# Define the output zip file name
```

```
output_zip_name = 'visdrone_exp114_results'
```

```
output_zip_path = f'/content/{output_zip_name}' # This will place the zip in the Colab environment
```

```
print(f"Creating zip archive of '{source_dir_to_zip}'...")
```

```
try:
```

```
# Create a zip archive of the specific experiment folder
shutil.make_archive(output_zip_path, 'zip', source_dir_to_zip)
print(f"Successfully created zip file: {output_zip_path}.zip")
```

```
# Optionally, copy it to Google Drive as well for persistent storage and easier download
```

```
google_drive_path = f'/content/drive/MyDrive/{output_zip_name}.zip'
```

```
shutil.copy(f'{output_zip_path}.zip', google_drive_path)
```

```
print(f"Also copied '{output_zip_name}.zip' to your Google Drive: {google_drive_path}")
```

```
print("You can now download this zip file from the Colab file browser (left sidebar -> 'Files' icon) or directly from your Google Drive")
except Exception as e:
```

```
    print(f"An error occurred while creating the zip file: {e}")
```

Creating zip archive of '/content/drive/MyDrive/yolo_runs/visdrone_exp114'...

Successfully created zip file: /content/visdrone_exp114_results.zip

Also copied 'visdrone_exp114_results.zip' to your Google Drive: /content/drive/MyDrive/visdrone_exp114_results.zip

You can now download this zip file from the Colab file browser (left sidebar -> 'Files' icon) or directly from your Google Drive

Reasoning: Now that the `visdrone_reduced.yaml` file is correctly configured with all 10 classes and the right paths, I will train the YOLO model using `yolo11n.pt` with these updated settings to detect all 10 classes.

```
from ultralytics import YOLO
import os
from google.colab import drive
```

```
# 1 Mount Google Drive (persistent storage)
drive.mount('/content/drive')

SAVE_DIR = "/content/drive/MyDrive/yolo_runs"
EXP_NAME = "visdrone_exp1"

# 2 Check if checkpoint exists (for resume)
checkpoint_path = f"{SAVE_DIR}/{EXP_NAME}/weights/last.pt"

if os.path.exists(checkpoint_path):
    print("🔄 Resuming from last checkpoint...")
    model = YOLO(checkpoint_path)
    resume_flag = True
else:
    print("🚀 Starting new training...")
    model = YOLO("yolo11s.pt") # Use small model
    resume_flag = False

# 3 Train
results = model.train(
    data='/content/fyp_Ai_extracted/FYP_AI-DRONES.1/dataset/VISdrone/VisDrone.yaml', # Using the newly created fixed YAML 1
    epochs=80,
    imgsz=850,
    batch=4,
    workers=2,
    patience=15,
    project=SAVE_DIR,
    name=EXP_NAME,
    save=True,
    save_period=5, # 🔥 Save every 5 epochs
    resume=resume_flag,
    device=0
)
```

bus	220	532	0.349	0.222	0.193	0.121
motor	131	251	0.72	0.558	0.661	0.499

Speed: 0.4ms preprocess, 5.9ms inference, 0.0ms loss, 3.4ms postprocess per image
Results saved to /content/drive/MyDrive/yolo_runs/visdrone_exp114

```
import os

# Check if pyserial is installed, if not, install it
try:
    import serial
except ImportError:
    print("pyserial not found, installing...")
    !pip install pyserial
    import serial # Import after installation

from ultralytics import YOLO
import cv2
import time
import numpy as np
from collections import deque
import threading
import queue

# =====
# CONFIGURATION
# =====
MODEL_PATH = "/content/drive/MyDrive/yolo_runs/visdrone_exp114/weights/best.pt"
CONF_THRESH = 0.3
MIN_AREA_RATIO = 0.008
MAX_AREA_RATIO = 0.6
FRAME_CONFIRM = 4

ZONES = {
    "left": (0.0, 0.33),
    "center": (0.33, 0.66),
    "right": (0.66, 1.0)
}

CLASS_WEIGHTS = {
    "person": 1.0,
    "car": 0.9,
    "truck": 0.8,
    "bus": 0.8,
    "motorcycle": 0.7,
    "bicycle": 0.6,
    "chair": 0.2,
    "table": 0.2
}

RISK_THRESHOLDS = {
    "low": 0.3,
    "medium": 0.6,
    "high": 1.0
}

# =====
# HARDWARE COMMUNICATION CLASS
# =====
class DroneHardware:
    def __init__(self, port='COM3', baud=9600):
        self.port = port
        self.baud = baud
        self.arduino = None
        self.connected = False
        self.command_queue = queue.Queue()

        # Motor control parameters (in microseconds)
        self.throttle = 1300 # Hover throttle
        self.pitch = 1500 # Neutral
        self.roll = 1500 # Neutral
        self.yaw = 1500 # Neutral

        # Initialize connection
        self.connect()

        # Start command sending thread
        self.running = True
        self.thread = threading.Thread(target=self._command_sender)
        self.thread.start()

    def connect(self):
        """Establish connection with Arduino"""
```

```

try:
    self.arduino = serial.Serial(self.port, self.baud, timeout=1)
    time.sleep(2) # Wait for Arduino reset
    self.connected = True
    print(f"[HARDWARE] Connected to Arduino on {self.port}")
except Exception as e:
    print(f"[HARDWARE] Connection failed: {e}")
    print("[HARDWARE] Running in SIMULATION mode")
    self.connected = False

def _command_sender(self):
    """Background thread to send commands"""
    while self.running:
        try:
            if self.connected:
                # Send current motor values every 50ms
                cmd = f"T{self.throttle}P{self.pitch}R{self.roll}Y{self.yaw}\n"
                self.arduino.write(cmd.encode())

                # Process queued commands
                if not self.command_queue.empty():
                    cmd = self.command_queue.get_nowait()
                    self._process_command(cmd)

            time.sleep(0.05) # 20Hz update rate

        except Exception as e:
            print(f"[HARDWARE ERROR] {e}")
            time.sleep(0.1)

def _process_command(self, command):
    """Process AI commands and convert to motor controls"""
    # Map AI commands to motor controls
    if command == "F": # Move forward
        self.pitch = 1600 # Lean forward
        self.roll = 1500 # Neutral roll
        print("[HARDWARE] Moving FORWARD")

    elif command == "L": # Turn left
        self.roll = 1400 # Lean left
        self.yaw = 1400 # Rotate left
        print("[HARDWARE] Turning LEFT")

    elif command == "R": # Turn right
        self.roll = 1600 # Lean right
        self.yaw = 1600 # Rotate right
        print("[HARDWARE] Turning RIGHT")

    elif command == "U": # Ascend
        self.throttle = min(self.throttle + 50, 1800)
        print("[HARDWARE] ASCENDING")

    elif command == "S": # Slow down
        self.pitch = 1450 # Slight forward
        print("[HARDWARE] SLOWING")

    elif command == "H": # Hover
        self.pitch = 1500
        self.roll = 1500
        self.yaw = 1500
        print("[HARDWARE] HOVERING")

    elif command == "X": # Emergency stop
        self.throttle = 1000
        self.pitch = 1500
        self.roll = 1500
        self.yaw = 1500
        print("[HARDWARE] EMERGENCY STOP")

def send_command(self, command):
    """Queue a command for execution"""
    if command in ["F", "L", "R", "U", "S", "H", "X"]:
        self.command_queue.put(command)
        return True
    return False

def get_status(self):
    """Get current hardware status"""
    return {
        "connected": self.connected,
        "throttle": self.throttle,
        "pitch": self.pitch,
        "roll": self.roll,

```

```

        "yaw": self.yaw
    }

def shutdown(self):
    """Safely shutdown hardware"""
    self.running = False
    if self.thread.is_alive():
        self.thread.join(timeout=2)

    # Send emergency stop before closing
    if self.connected:
        try:
            self.arduino.write(b"X\n")
            time.sleep(0.1)
            self.arduino.close()
        except:
            pass

    print("[HARDWARE] Shutdown complete")

# =====
# ENHANCED DRONE CONTROLLER
# =====
class AutonomousDrone:
    def __init__(self):
        # AI Vision
        print("[INFO] Loading YOLO model...")
        self.model = YOLO(MODEL_PATH)

        # Hardware Interface
        self.hardware = DroneHardware(port='COM3') # Change to your port

        # Camera
        self.cap = cv2.VideoCapture(0)
        if not self.cap.isOpened():
            print("[ERROR] Camera not accessible")
            exit()

        # State Management
        self.state = {
            "confirmed_frames": 0,
            "last_command": "F",
            "command_history": deque(maxlen=8),
            "obstacle_grid": np.zeros((3, 3)),
            "stuck_counter": 0,
            "fps": 0,
            "frame_count": 0,
            "last_risk_time": time.time()
        }

        # Safety parameters
        self.safe_throttle = 1300 # Just above hover
        print("[INFO] Autonomous Drone System Ready")

    def get_zone(self, x, w):
        """Convert x position to zone"""
        nx = x / w
        for name, (a, b) in ZONES.items():
            if a <= nx <= b:
                return name
        return "center"

    def calculate_risk(self, cls, area_ratio, zone, conf):
        """Calculate risk score for detection"""
        if cls not in CLASS_WEIGHTS:
            return 0.0
        size_factor = np.log1p(area_ratio * 100)
        pos_factor = 1.5 if zone == "center" else 1.0
        return min(CLASS_WEIGHTS[cls] * size_factor * pos_factor * conf, 3.0)

    def update_obstacle_grid(self, detections, w, h):
        """Update spatial obstacle grid"""
        self.state["obstacle_grid"] = np.zeros((3, 3))
        for d in detections:
            gx = min(int((d["center_x"] / w) * 3), 2)
            gy = min(int((d["center_y"] / h) * 3), 2)
            self.state["obstacle_grid"][gy, gx] += d["risk"]

    def choose_direction(self):
        """Choose safest direction based on obstacle grid"""
        left = self.state["obstacle_grid"][:, 0].sum()
        center = self.state["obstacle_grid"][:, 1].sum()
        right = self.state["obstacle_grid"][:, 2].sum()

```



```

print(f"[NAV] Obstacle densities - L:{left:.2f} C:{center:.2f} R:{right:.2f}")

if left <= right and left <= center * 0.8:
    return "L"
elif right <= left and right <= center * 0.8:
    return "R"
else:
    return "U"

def process_frame(self, frame):
    """Process single frame and make navigation decision"""
    h, w = frame.shape[:2]
    total_risk = 0.0
    center_risk = 0.0
    detections = []

    # =====
    # OBJECT DETECTION
    # =====
    try:
        results = self.model(frame, conf=CONF_THRESH, verbose=False)
    except Exception as e:
        print(f"[ERROR] Model inference failed: {e}")
        results = []

    for r in results:
        if not getattr(r, "boxes", None):
            continue

        for box in r.boxes:
            # Extract coordinates safely
            try:
                coords = box.xyxy[0]
                if hasattr(coords, 'cpu'):
                    coords = coords.cpu().numpy()
                x1, y1, x2, y2 = map(int, coords)
            except Exception:
                continue

            cls = self.model.names[int(box.cls[0])]
            conf = float(box.conf[0])

            # 🕒 DRAW ALL YOLO DETECTIONS (DEBUG)
            cv2.rectangle(frame, (x1, y1), (x2, y2), (120, 120, 120), 1)
            cv2.putText(frame, f"{cls} {conf:.2f}",
                        (x1, y2 + 12),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.4,
                        (180, 180, 180), 1)

            # Only navigation-relevant classes continue
            if cls not in CLASS_WEIGHTS:
                continue

            area_ratio = ((x2 - x1) * (y2 - y1)) / float(h * w)
            if area_ratio < MIN_AREA_RATIO or area_ratio > MAX_AREA_RATIO:
                continue

            cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
            zone = self.get_zone(cx, w)

            risk = self.calculate_risk(cls, area_ratio, zone, conf)

            detections.append({
                "center_x": cx,
                "center_y": cy,
                "risk": risk
            })

            total_risk += risk
            if zone == "center":
                center_risk += risk

            # 🔥 RISK VISUALIZATION
            color = (0, 255, 0)
            if risk > 1.2:
                color = (0, 0, 255)
            elif risk > 0.6:
                color = (0, 165, 255)

            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
            cv2.putText(frame, f"R:{risk:.2f}",

```

```

        (x1, y1 - 6),
        cv2.FONT_HERSHEY_SIMPLEX, 0.5,
        color, 2)

# =====
# UPDATE STATE (AFTER ALL DETECTIONS)
# =====
self.update_obstacle_grid(detections, w, h)

# Stability logic
if total_risk > RISK_THRESHOLDS["low"]:
    self.state["confirmed_frames"] = min(self.state["confirmed_frames"] + 1, FRAME_CONFIRM * 2)
else:
    self.state["confirmed_frames"] = max(self.state["confirmed_frames"] - 1, 0)

environment_blocked = total_risk > RISK_THRESHOLDS["low"]

# =====
# NAVIGATION DECISION
# =====
if environment_blocked and self.state["confirmed_frames"] >= FRAME_CONFIRM:
    direction = self.choose_direction()
    command = direction
    action_text = f"AVOID {direction}"
    action_color = (0, 0, 255)
else:
    command = "F"
    action_text = "MOVING FORWARD"
    action_color = (0, 255, 0)

# Stuck prevention
if command == self.state["last_command"] and environment_blocked:
    self.state["stuck_counter"] += 1
else:
    self.state["stuck_counter"] = 0

if self.state["stuck_counter"] > 3 * max(self.state["fps"], 1):
    command = "U"
    action_text = "ESCAPE ASCEND"
    action_color = (255, 0, 255)
    self.state["stuck_counter"] = 0
    print("[WARNING] Executing escape maneuver")

# Send command to hardware
self.hardware.send_command(command)

# Update state
self.state["last_command"] = command
self.state["command_history"].append(command)

# =====
# VISUALIZATION
# =====
# Main action display
cv2.putText(frame, f"CMD: {command}", (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 1, action_color, 2)

# Status panel
status_y = 75
status_lines = [
    f"Risk: {total_risk:.2f}",
    f"FPS: {self.state['fps']}",
    f"Stability: {self.state['confirmed_frames']}/{FRAME_CONFIRM}",
    f"Hardware: {'CONNECTED' if self.hardware.connected else 'SIMULATION'}"
]

for line in status_lines:
    cv2.putText(frame, line, (20, status_y),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)
    status_y += 25

# Draw navigation zones
for zone_name, (zone_start, zone_end) in ZONES.items():
    x1_z, x2_z = int(zone_start * w), int(zone_end * w)
    zone_color = (100, 100, 100) if zone_name != "center" else (255, 255, 0)
    cv2.rectangle(frame, (x1_z, 0), (x2_z, h), zone_color, 1)

# Hardware status (right side)
hw_status = self.hardware.get_status()
hw_text = f"Thr:{hw_status['throttle']} Pit:{hw_status['pitch']}"
cv2.putText(frame, hw_text, (w - 200, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (200, 200, 255), 1)

```

```

        return frame, command, total_risk

def run(self):
    """Main execution loop"""
    fps_timer = time.time()

    try:
        while True:
            # Calculate FPS
            self.state["frame_count"] += 1
            if time.time() - fps_timer >= 1:
                self.state["fps"] = self.state["frame_count"]
                self.state["frame_count"] = 0
                fps_timer = time.time()

            # Capture frame
            ret, frame = self.cap.read()
            if not ret:
                print("[WARNING] Frame capture failed")
                time.sleep(0.1)
                continue

            # Process frame
            processed_frame, command, risk = self.process_frame(frame)

            # Display
            cv2.imshow("Autonomous Drone - AI Navigation", processed_frame)

            # Console output (reduced frequency)
            current_time = time.time()
            if command != self.state["last_command"] or current_time - self.state["last_risk_time"] > 2:
                print(f"[{time.strftime('%H:%M:%S')}] CMD:{command} | Risk:{risk:.2f}")
                self.state["last_risk_time"] = current_time

            # Handle keyboard input
            key = cv2.waitKey(1) & 0xFF
            if key == ord('q'):
                print("[INFO] Manual shutdown requested")
                break
            elif key == ord('h'):
                self.hardware.send_command("H")
                print("[INFO] Hover command sent")
            elif key == ord('x'):
                self.hardware.send_command("X")
                print("[INFO] Emergency stop sent")
            elif key == ord('d'):
                print(f"[DEBUG] Grid:\n{self.state['obstacle_grid']}")
                print(f"[DEBUG] Cmd history: {list(self.state['command_history'])}")

        except KeyboardInterrupt:
            print("\n[INFO] Interrupted by user")

        finally:
            self.shutdown()

def shutdown(self):
    """Clean shutdown"""
    print("[INFO] Shutting down system...")

    # Emergency stop
    self.hardware.send_command("X")

    # Release resources
    self.cap.release()
    cv2.destroyAllWindows()
    self.hardware.shutdown()

    print("[INFO] System shutdown complete")

# =====
# ARDUINO CODE (Upload this separately)
# =====
"""
// ARDUINO CODE: flight_controller.ino
#include <Servo.h>

Servo motor_FL, motor_FR, motor_BL, motor_BR;

// Default neutral positions (in microseconds)
int throttle = 1000; // STOP
int pitch = 1500;    // NEUTRAL
int roll = 1500;     // NEUTRAL
int yaw = 1500;      // NEUTRAL

```

```

void setup() {
  Serial.begin(9600);

  // Attach motors to pins (adjust based on your wiring)
  motor_FL.attach(9);
  motor_FR.attach(10);
  motor_BL.attach(11);
  motor_BR.attach(12);

  // Initialize to STOP
  updateMotors();

  Serial.println("Arduino Flight Controller Ready");
}

void loop() {
  if (Serial.available()) {
    String input = Serial.readStringUntil('\n');
    processCommand(input);
  }
}

void processCommand(String cmd) {
  if (cmd.length() >= 12 && cmd[0] == 'T') {
    // Format: T1300P1500R1500Y1500
    throttle = cmd.substring(1, 5).toInt();
    pitch = cmd.substring(6, 10).toInt();
    roll = cmd.substring(11, 15).toInt();
    yaw = cmd.substring(16, 20).toInt();

    // Safety limits
    throttle = constrain(throttle, 1000, 2000);
    pitch = constrain(pitch, 1200, 1800);
    roll = constrain(roll, 1200, 1800);
    yaw = constrain(yaw, 1200, 1800);

    updateMotors();

    // Echo back for verification
    Serial.print("OK:T");
    Serial.print(throttle);
    Serial.print(" P");
    Serial.print(pitch);
    Serial.print(" R");
    Serial.print(roll);
    Serial.print(" Y");
    Serial.println(yaw);
  }
}

void updateMotors() {
  // Basic mixing for quadcopter (simplified)
  motor_FL.writeMicroseconds(throttle - pitch + roll - yaw);
  motor_FR.writeMicroseconds(throttle - pitch - roll + yaw);
  motor_BL.writeMicroseconds(throttle + pitch + roll + yaw);
  motor_BR.writeMicroseconds(throttle + pitch - roll - yaw);
}

"""

# =====
# MAIN EXECUTION
# =====
if __name__ == "__main__":
    print("=" * 50)
    print("AUTONOMOUS DRONE NAVIGATION SYSTEM")
    print("FYP - BATCH SP23-BCS")
    print("=" * 50)

    # Run the system
    drone = AutonomousDrone()
    drone.run()

```

```

=====
AUTONOMOUS DRONE NAVIGATION SYSTEM
FYP - BATCH SP23-BCS
=====
[INFO] Loading YOLO model...
[HARDWARE] Connection failed: [Errno 2] could not open port COM3: [Errno 2] No such file or directory: 'COM3'
[HARDWARE] Running in SIMULATION mode
[ERROR] Camera not accessible
[INFO] Autonomous Drone System Ready

```

[illegible]